StanfordQuadruped 项目解读教程

面向 StanfordQuadruped 项目本身的源码解读、控制链路拆解与工程阅读指南

从 run_robot.py 到 gait / stance / swing / IK / PWM 的独立项目教程

阅读主线:
入口脚本 -> Command -> 状态机 -> 步态 -> 足端 -> IK -> 舵机输出

学习导航

建议先通读主线，再按章节动手复现，再回到仓库代码做映射。

- 1. 先给结论：这是一个什么风格的控制器
- 2. 仓库总览：先认目录，再读代码
- 3. 从 `run_robot.py` 开始：整个项目的总入口
- 4. 输入层：`JoystickInterface` 怎样把手柄变成 `Command`
  - 4.1 连续量映射
  - 4.2 离散事件映射
- 5. 状态层：`State` 和 `BehaviorState` 存了什么
  - 5.1 一个仓库里的小实现细节
- 6. 控制核心：`Controller.run()` 才是全项目的大脑
- 7. 步态调度：`GaitController` 怎么决定哪条腿摆、哪条腿撑
  - 7.1 它的 gait 不是连续相位函数，而是离散相位表
  - 7.2 时间参数是怎么来的
- 8. 支撑相控制：`StanceController` 其实做得很朴素
  - 8.1 这不是动力学支撑控制，而是几何支撑控制
  - 8.2 它的优点和局限
- 9. 摆动相控制：`SwingController` 采用的是 Raibert 风格着地点
  - 9.1 着地点选择
  - 9.2 摆腿高度
  - 9.3 足端前向推进
- 10. `TROT` 状态如何把各模块真正串起来
  - 10.1 `step_gait()` 到底返回什么
  - 10.2 这里最重要的数据结构就是 `(3, 4)`
- 11. 姿态补偿：这个项目不是纯开环，它有 IMU 倾斜反馈
  - 11.1 命令姿态旋转
  - 11.2 IMU 倾斜补偿
  - 11.3 `REST` 模式里还有一个平滑 yaw
  - 11.4 这里的反馈还停留在哪一层
- 12. 逆运动学：`pupper/Kinematics.py` 是从足端到舵机的桥
  - 12.1 单腿 IK 做了什么
  - 12.2 为什么 IK 是这个仓库最关键的 “ 桥 ”
- 13. 硬件输出：`joint_angles` 怎么变成 PWM
  - 13.1 从角度到 PWM 的公式
  - 13.2 每条腿符号为什么不一样

- ◦ 13.3 标定文件是机器生成的
- • 14. 配置：这个项目几乎所有行为都被 `Configuration` 控着
- ◦ 14.1 如果你只想让机器人“ 走起来 ”，先看这些参数
- ◦ 14.2 如果你想让它“ 走得更稳 ”，优先看这些参数
- ◦ 14.3 如果你想让 IK 和机械结构对齐，先看这些参数
- • 15. 部署与外部依赖：真正跑实机时还缺哪些东西
- • 16. 这个项目最适合怎样继续改
- ◦ 16.1 想接导航或高层策略
- ◦ 16.2 想接 MuJoCo / Gazebo
- ◦ 16.3 想加更复杂控制器
- • 17. 代码阅读时最该注意的几个“ 项目特点 ”
- ◦ 17.1 它是强几何、弱动力学的项目
- ◦ 17.2 它非常适合教学，但也带着历史痕迹
- ◦ 17.3 真正的主干文件没有很多
- • 18. 和通用四足控制教程怎么对照着读
- • 19. 最后给一个最稳的项目阅读顺序
- • 20. 先把四个核心变量的流向记清楚
- ◦ 20.1 总链路先看一遍
- ◦ 20.2 `Command` 的流向
- ◦ 20.3 `State` 的流向
- ◦ 20.4 `foot_locations` 的流向
- ◦ 20.5 `joint_angles` 的流向
- ◦ 20.6 先判断你到底该改“ 输入 ”还是改“ 步态 ”
- • 附录 A：最短路径版总结
- • 附录 B：你读这个项目时最该盯住的变量
- • 附录 C：这个项目最适合拿来学什么

StanfordQuadruped 项目解读教程

这份教程面向 `refcode/stanford_quadruped`，目标不是泛泛讲四足机器人，而是把这个仓库本身拆开讲清楚：它从哪里读输入、怎样做状态切换、怎样生成足端目标、怎样做逆运动学、最后又怎样把关节角发给舵机。

> 如果你已经读过 `docs/quadruped_motion_control_tutorial/quadruped_motion_control_tutorial.pdf`，可以把这份文档理解成“把通用运动控制理论，落到 StanfordQuadruped 这份具体代码上的项目解读版”。

1. 先给结论：这是一个什么风格的控制器

StanfordQuadruped 不是 MPC，也不是 WBC，更不是全身动力学求解器。

它更接近下面这类工程结构：

```
手柄输入
 -> 命令对象 Command
 -> 行为状态机
 -> 步态调度
 -> 支撑相 / 摆动相足端规划
 -> 四腿逆运动学
 -> 关节角
 -> PWM / 舵机输出
```

它的关键词可以概括为：

- 以 `run_robot.py` 为主循环入口
- 以 `src/Controller.py` 为控制核心
- 以 `src/Gaits.py`、`src/StanceController.py`、`src/SwingLegController.py` 组织腿级逻辑
- 以 `pupper/Kinematics.py` 做足端到关节角的解析逆解
- 以 `pupper/HardwareInterface.py` 把关节角转换成舵机 PWM
- 以 `pupper/Config.py` 统一存放步态、机体、几何、控制参数

从控制分层上说，它是一个很典型的：

- 低阶：位置型舵机控制项目
- 中阶：基于足端轨迹与 IK 的步态控制项目
- 高阶：带简单姿态补偿的手柄遥控四足项目

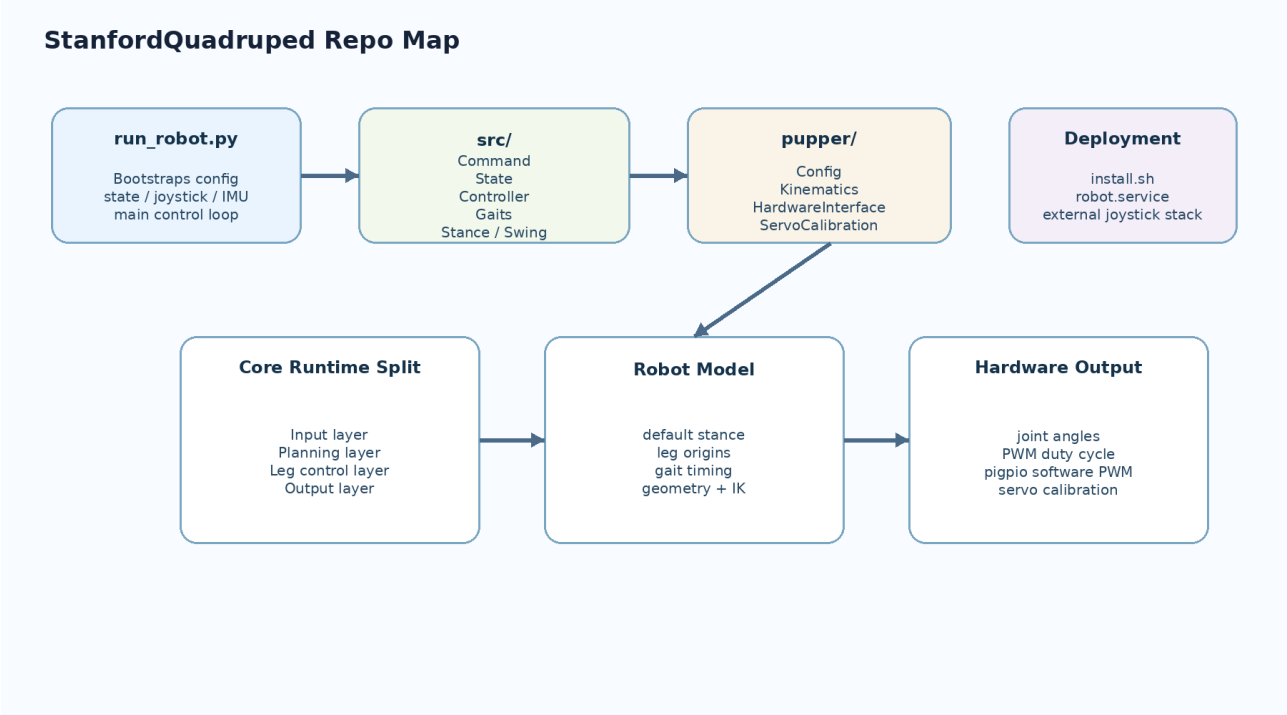
这意味着它非常适合拿来做三件事：

- 入门四足机器人控制主链
- 学“命令 -> 足端 -> IK -> 舵机”的完整闭环
- 在现有仓库上继续加自己的步态、接口、仿真后端

不太适合直接拿来做的事是：

- 高速动态跑跳的动力学最优控制
- 严格的接触力分配
- 全身约束优化

2. 仓库总览：先认目录，再读代码



StanfordQuadruped 仓库模块图

对这个项目，最推荐的第一眼目录图是：

```
run_robot.py
  -> 运行时主循环入口

src/
  -> Command / State / Controller / Gaits / Stance / Swing

pupper/
  -> Config / Kinematics / HardwareInterface / ServoCalibration

install.sh + robot.service
  -> 依赖安装与开机启动
```

你可以把仓库分成 4 层：

- 入口层：`run_robot.py`
- 控制层：`src/`
- 机器人模型与硬件层：`pupper/`
- 部署层：`install.sh`、`robot.service`

另外还有一条容易被忽略的旁支：

- `woofer/`

这说明这个仓库历史上不只服务 `Pupper`，还同时承载过 `Woofer` 这条更大机器人的代码分支。它和 `pupper/` 的差异很大：

```
pupper/
 -> 舵机 + pigpio + PWM

woofer/
 -> ODrive + 编码器计数 + 更大几何尺寸
```

所以这份教程默认聚焦的是：

```
run_robot.py + src/ + pupper/
```

而不是 `woofer/` 支线。

对应到“ 阅读顺序 ”，最稳的是：

```
先看 run_robot.py
 -> 再看 Controller.py
 -> 再看 Gaits / Stance / Swing
 -> 再看 Kinematics.py
 -> 最后看 HardwareInterface.py 和 Config.py
```

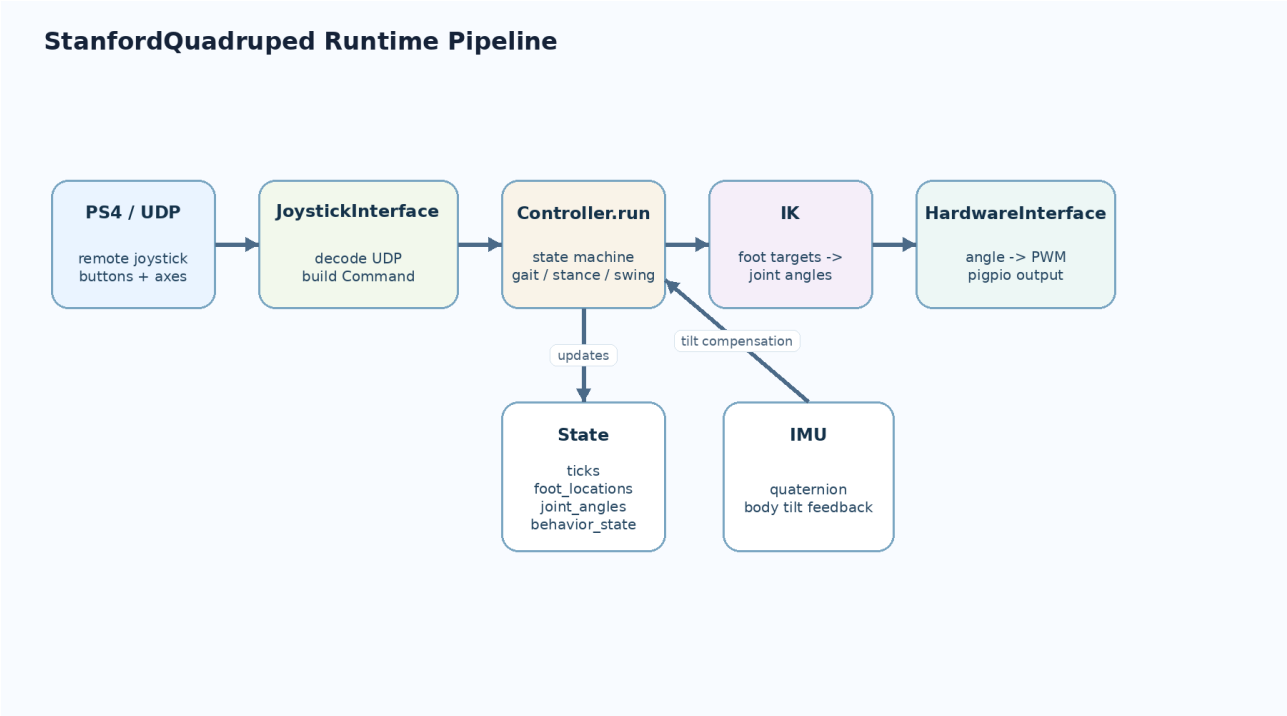
3. 从 `run_robot.py` 开始：整个项目的总入口

这个仓库的运行时总入口就是 `refcode/stanford_quadruped/run_robot.py`。

它做的事情非常集中：

- 创建配置对象 `Configuration()`
- 创建硬件接口 `HardwareInterface()`
- 可选创建 IMU 句柄
- 创建控制器 `Controller(...)`
- 创建状态对象 `State()`
- 创建手柄接口 `JoystickInterface(config)`
- 进入固定周期循环

运行时主链可以压缩成：



StanfordQuadruped 运行时主链路

```
config = Configuration()
hardware = HardwareInterface()
controller = Controller(config, four_legs_inverse_kinematics)
state = State()
joystick = JoystickInterface(config)

while robot_is_active:
    command = joystick.get_command(state)
    state.quat_orientation = imu.read_orientation() or [1, 0, 0, 0]
    controller.run(state, command)
    hardware.set_actuator_postions(state.joint_angles)
```

这里最关键的理解是：

- `run_robot.py` 自己几乎不做控制算法
- 它只负责把输入、状态、控制器、硬件接口串起来
- 真正的运动控制逻辑集中在 `Controller.run()`

这个文件还有两个很重要的工程特征：

- 它用 `config.dt` 控制循环周期，默认是 `0.01s`
- 文件末尾直接调用 `main()`，没有 `if __name__ == "__main__"` 防护

第二点意味着：

- 如果你直接 import 这个文件，它也会立即开始运行主循环
- 所以后续如果你要做仿真接入、单元测试、模块复用，通常第一步就会先把入口改成可导入形式

还有两个运行时细节也很值得单独记住：

外层 while
-> 等待 L1 激活
-> 进入运行循环

-> 再按一次 L1 退出到等待态

以及：

```
if now - last_loop < config.dt:
    continue
```

这是一种非常直接的忙等式定时循环。

它的优点是简单；它的代价是：

- 会空转占 CPU
- 不适合作为更复杂系统里的高质量调度方案

另外，IMU 也是可选的：

```
use_imu = False -> 直接使用单位四元数 [1, 0, 0, 0]
```

这意味着控制器本身支持：

- 有 IMU 的姿态补偿运行
- 无 IMU 的纯几何 gait 运行

4. 输入层：`JoystickInterface` 怎样把手柄变成 `Command`

`src/JoystickInterface.py` 的职责非常明确：

- 从 UDP 订阅手柄消息
- 把按钮和摇杆映射成项目内部统一的 `Command`
- 检测边沿触发事件，比如切步态、切激活、起跳

它生成的命令对象定义在 `src/Command.py`：

```
horizontal_velocity = [vx, vy]
yaw_rate
height
pitch
roll
hop_event
trot_event
activate_event
```

也就是说，这个项目的上层控制输入最后被压缩成：

```
cmd = [vx, vy, yaw_rate, height, pitch, roll] + 离散按钮事件
```

4.1 连续量映射

在手柄映射里，最重要的连续量是：

- `ly -> x_vel`
- `lx -> y_vel`

- `rx -> yaw_rate`
- `ry -> pitch`
- `dpady -> body height`
- `dpadx -> roll`

更准确地说，它在代码里做的是：

```
x_vel  = ly * max_x_velocity
y_vel  = lx * -max_y_velocity
yaw    = rx * -max_yaw_rate
pitch  = ry * max_pitch
height = state.height - dt * z_speed * dpady
roll   = state.roll + dt * roll_speed * (-dpadx)
```

这里有两个细节非常值得注意：

- pitch 不是直接赋值，而是过了 `deadband` 和一阶限速滤波
- height 和 roll 是“ 在当前状态上积分 ”，不是绝对值命令

所以它并不是“ 手柄值直接进控制器 ”，中间已经做了基本的命令整形。

4.2 离散事件映射

按钮的作用是：

```
L1 -> activate_event
R1 -> trot_event
X  -> hop_event
```

而且是边沿触发：

```
event = (current_button == 1 and previous_button == 0)
```

这意味着按钮是“ 切换模式 ”，不是“ 持续保持某模式 ”。

5. 状态层：`State` 和 `BehaviorState` 存了什么

`src/State.py` 里真正长期流动的状态不多，但都很关键：

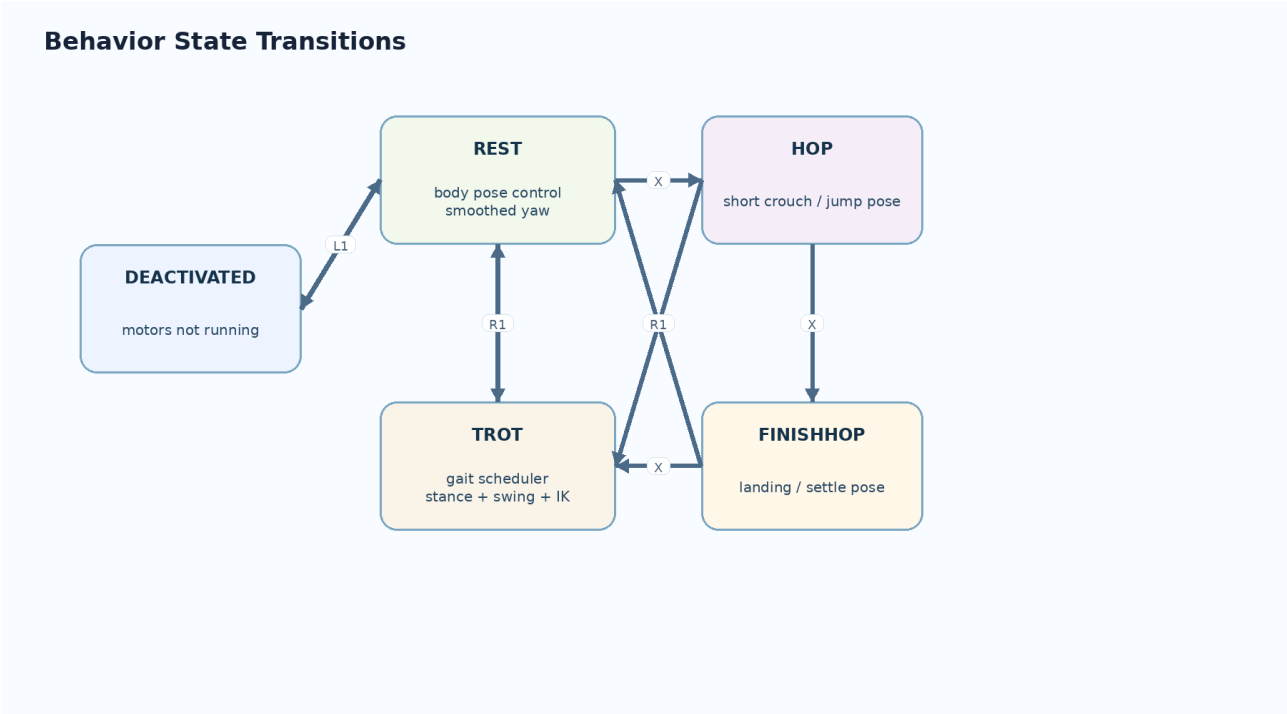
- 当前速度/姿态命令记忆
- 当前行为状态 `behavior_state`
- 当前时间计数 `ticks`
- 当前足端位置 `foot_locations`
- 当前关节角 `joint_angles`

它的核心行为状态是：

```
DEACTIVATED
REST
TROT
HOP
```

FINISHHOP

状态切换关系可以读成：



StanfordQuadruped 行为状态切换图

DEACTIVATED <-> REST
REST <-> TROT
REST -> HOP -> FINISHHOP -> REST
TROT -> HOP -> FINISHHOP -> TROT 或 REST

其中映射表直接写在 `Controller.__init__()` 里：

- `activate_transition_mapping`
- `trot_transition_mapping`
- `hop_transition_mapping`

这套写法非常工程化：

- 好处是简单直接
- 不好处是状态增多后会很快变乱

如果以后你要加：

- Crawl
- Bound
- Sit / StandUp
- Recovery

那就很适合把这块重构成一个显式状态机类。

5.1 一个仓库里的小实现细节

这个仓库里还有一个小“历史味”很重的点：

- `State.__init__()` 里没有预先定义 `quat_orientation`
- 但 `run_robot.py` 会在循环里动态写入 `state.quat_orientation`

这不影响当前运行，但对类型检查、代码整洁度和后续扩展都不太友好。

6. 控制核心：`Controller.run()` 才是全项目的大脑

如果你只准看一个文件，那就先看 `src/Controller.py`。

它干的事，可以压缩成：

```
先根据按钮更新 behavior_state
  -> 再按状态分支执行 REST / TROT / HOP / FINISHHOP
  -> 最后把 foot_locations 和 joint_angles 写回 state
```

更具体一点：

```
if activate_event:
    切 DEACTIVATED / REST
elif trot_event:
    切 REST / TROT
elif hop_event:
    切 HOP 流程

if state == TROT:
    gait + stance + swing + IK + IMU compensation
elif state == HOP:
    直接给一个更低的足端姿态
elif state == FINISHHOP:
    给另一个收尾姿态
elif state == REST:
    站姿 + body pitch/roll/yaw 调整 + IK
```

这说明它本质上是一个“模式切换 + 几何控制器”，而不是单一 gait 函数。

7. 步态调度：`GaitController` 怎么决定哪条腿摆、哪条腿撑

`src/Gaits.py` 很短，但意义很大。

它只做三件事：

- `phase_index(ticks)`：当前处于第几个 gait phase
- `subphase_ticks(ticks)`：当前 phase 已经走了多少 tick
- `contacts(ticks)`：4 条腿里谁是 stance，谁是 swing

7.1 它的 gait 不是连续相位函数，而是离散相位表

`pupper/Config.py` 里给出了：

```
num_phases = 4
contact_phases =
[
 [1, 1, 1, 0],
 [1, 0, 1, 1],
 [1, 0, 1, 1],
 [1, 1, 1, 0],
]
```

如果按腿序：

```
0 = front-right
1 = front-left
2 = back-right
3 = back-left
```

那么 4 个 phase 可以读成：

```
phase 0: 四腿都着地
phase 1: FL + BR 摆动，FR + BL 支撑
phase 2: 四腿都着地
phase 3: FR + BL 摆动，FL + BR 支撑
```

这就是一个非常典型的对角小跑 trot。

7.2 时间参数是怎么来的

配置里默认：

```
dt = 0.01
overlap_time = 0.10
swing_time   = 0.15
```

所以可以直接算出：

```
overlap_ticks = 10
swing_ticks   = 15
phase_ticks   = [10, 15, 10, 15]
phase_length  = 50
```

这意味着完整 gait 周期大约是：

```
0.50 秒
```

这个节律对于一个低成本舵机四足来说是很合理的：

- 不激进
- 不追求极限动态
- 稳定、易调、易讲清楚

8. 支撑相控制：`StanceController` 其实做得很朴素

`src/StanceController.py` 的核心思想可以压成一句话：

> 机体想往前走，就让支撑脚在机体系里往后“ 刨地 ”。

它的核心增量模型可以写成：

```
v_xy =
[
  -cmd_vx,
  -cmd_vy,
  (state.height - z) / z_time_constant
]

delta_p = v_xy * dt
delta_R = Rz(-yaw_rate * dt)
foot_next = delta_R @ foot_now + delta_p
```

这里有 3 个层面的含义：

- x、y 方向：支撑脚相对机体向反方向移动
- z 方向：把当前脚高拉回到目标机身高度
- yaw 方向：绕机体中心反向旋转支撑脚

8.1 这不是动力学支撑控制，而是几何支撑控制

它没有显式算：

- 接触力
- 质心加速度
- ZMP
- friction cone

它只是用一个非常干净的几何近似去产生支撑相足端位移。

这也是这个仓库很适合教学的原因：

- 逻辑短
- 数据流清楚
- 很容易看懂“ 速度命令到底去哪了 ”

8.2 它的优点和局限

优点：

- 参数少
- 运行轻量
- 行为直观

局限：

- 对高速动态工况不够
- 对剧烈机身姿态扰动补偿有限
- 没有显式接触力控制

9. 摆动相控制：`SwingController` 采用的是 Raibert 风格着地点

`src/SwingLegController.py` 的核心分成三步：

- 算 touchdown location
- 算 swing height
- 用剩余时间把当前脚送到 touchdown 位置

9.1 着地点选择

着地点模型可以写成：

```
delta_p = alpha * stance_ticks * dt * horizontal_velocity
theta   = beta  * stance_ticks * dt * yaw_rate
touchdown = Rz(theta) @ default_stance[:, leg] + delta_p
```

它的直觉是：

- 你在 stance 里让脚往后走了多远
- swing 里就应该把脚送回前面相应的位置

这就是 Raibert 风格思想在这个小项目里的落地版。

9.2 摆腿高度

摆腿高度不是五次多项式，而是一个简单三角波：

```
if swing_phase < 0.5:
    z = swing_phase / 0.5 * z_clearance
else:
    z = z_clearance * (1 - (swing_phase - 0.5) / 0.5)
```

这个选择非常符合项目气质：

- 实现简单
- 可解释性强
- 对教学很友好

9.3 足端前向推进

真正给下一时刻的脚位置时，它用的是：

```
time_left = dt * swing_ticks * (1 - swing_prop)
v = (touchdown - foot_now) / time_left
delta = v * dt
z = swing_height + command.height
foot_next = [x_now, y_now, 0] + delta_xy + [0, 0, z]
```

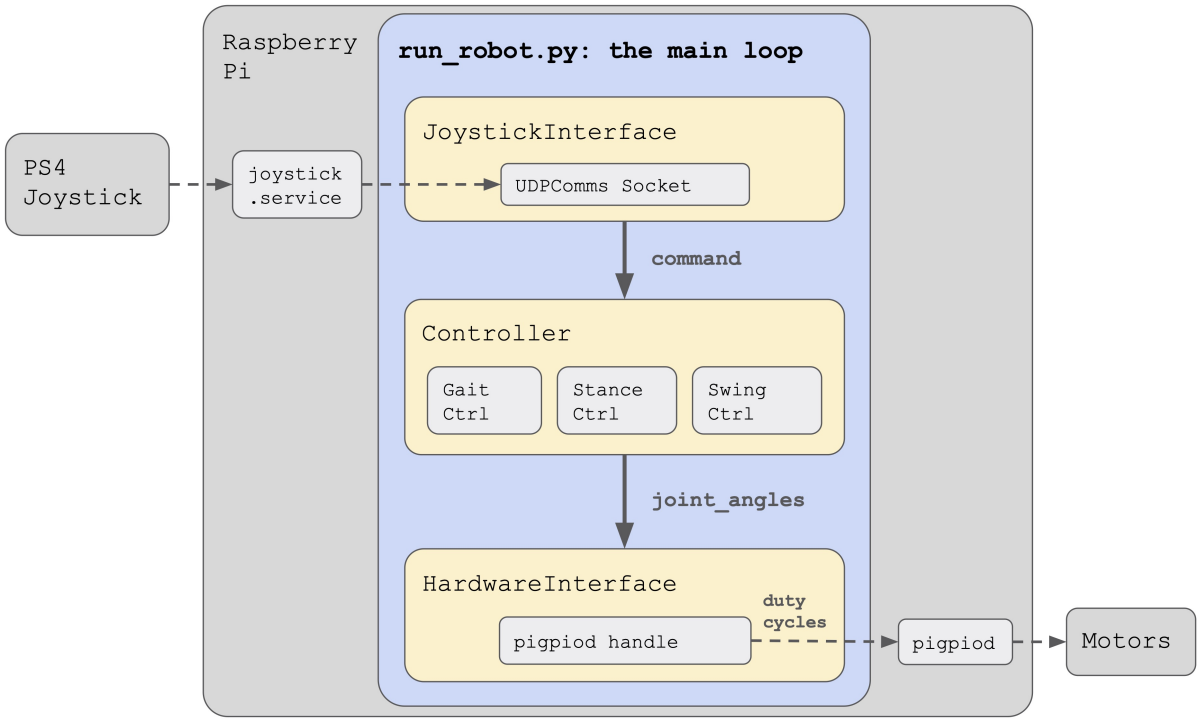
所以它不是一次性把脚“跳”到 touchdown，而是每个控制周期都重新算一小步。

10. `TROT` 状态如何把各模块真正串起来

在 `Controller.run()` 的 `TROT` 分支里，主链是：

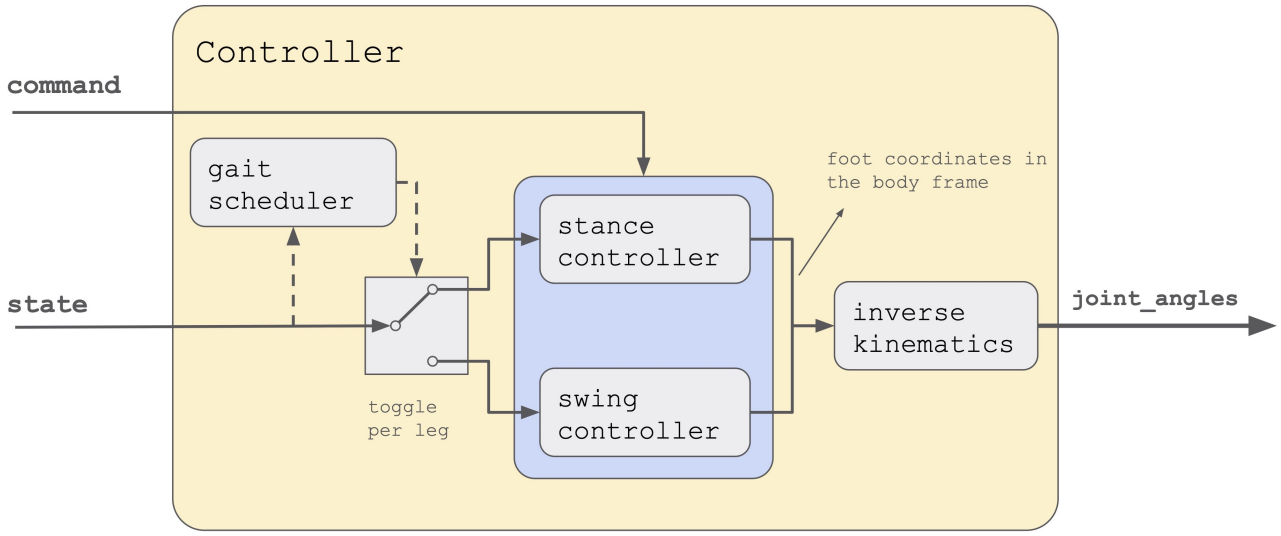
```
step_gait()
  -> 得到新的 foot_locations 与 contact_modes
  -> 叠加命令的 roll / pitch 机身旋转
  -> 用 IMU 做机身倾斜补偿
  -> inverse_kinematics()
  -> joint_angles
```

项目自带的总览图就是在讲这件事：



项目自带总览图

而控制器结构图则更细：



项目自带控制器图

10.1 `step_gait()` 到底返回什么

`step_gait()` 会对四条腿逐条判断：

```
if contact_mode == 1:
    用 StanceController
else:
    用 SwingController
```

最后返回：

- `new_foot_locations`：形状 `(3, 4)` 的足端位置矩阵
- `contact_modes`：形状 `(4,)` 的接触标志

这里还有一个阅读时容易忽略的小点：

- `contact_modes` 确实被算出来了
- 但在当前主路径里，它没有被长期保存到 `state`
- 它更多是本周期内的腿级判定结果

这说明这个仓库的接触语义更偏“步态表驱动”，而不是“估计器反馈驱动”。

10.2 这里最重要的数据结构就是 `(3, 4)`

这个仓库很多核心量都按：

```
rows = [x, y, z]
cols = [FR, FL, BR, BL]
```

组织，也就是：

```
matrix shape = (3, 4)
```

比如：

- `default_stance`
- `foot_locations`
- `joint_angles`

只要这一点没看懂，读代码就会一直别扭。

11. 姿态补偿：这个项目不是纯开环，它有 IMU 倾斜反馈

很多人第一次看这个仓库，以为它完全是开环 gait。

其实不是。

在 `TROT` 状态里，它做了两个层次的机身旋转：

11.1 命令姿态旋转

先根据手柄命令做：

```
rotated_foot_locations = R(roll_cmd, pitch_cmd, 0) @ foot_locations
```

这表示：

- 用户希望机身有一定 roll/pitch
- 那就把足端目标按相反几何关系旋转过去

11.2 IMU 倾斜补偿

然后它从四元数中解出：

- roll
- pitch
- yaw

再对 roll/pitch 做：

```
roll_comp  = 0.8 * clip(roll,  -0.4, 0.4)
pitch_comp = 0.8 * clip(pitch, -0.4, 0.4)
rmat = R(roll_comp, pitch_comp, 0)
rotated_foot_locations = rmat.T @ rotated_foot_locations
```

这一步的含义非常直接：

- 如果机身真的歪了
- 那就让足端目标反向修一点
- 让站姿和 trot 更稳

它不是完整状态估计器，但它已经不是“完全不看反馈”的纯开环。

11.3 `REST` 模式里还有一个平滑 yaw

在 `REST` 模式里，`yaw` 不是直接跳到目标，而是：

```
smoothed_yaw += dt * clipped_first_order_filter(...)
```

然后把 `roll / pitch / smoothed_yaw` 一起作为机身旋转施加到默认站姿上。

所以 `REST` 更像：

```
一个静态站姿姿态调节器
 -> 允许用户调 body height / roll / pitch / yaw
 -> 再通过 IK 把它变成关节角
```

11.4 这里的反馈还停留在哪一层

虽然用了 IMU，但这里的反馈层次仍然比较浅：

```
IMU 四元数
 -> roll / pitch
 -> 足端几何补偿
```

它没有进一步做：

- 速度估计
- 接触状态估计
- 质心状态闭环
- 接触力闭环

所以这个项目里的“反馈控制”更准确地说是：

- 姿态倾斜补偿
- 不是完整状态估计驱动的动力学控制

12. 逆运动学：`pupper/Kinematics.py` 是从足端到舵机的桥

这个项目真正把“控制算法输入”变成“舵机目标”的关键桥梁，就是 IK。

它的总入口是：

```
four_legs_inverse_kinematics(r_body_foot, config)
```

其中输入是：

```
r_body_foot.shape == (3, 4)
```

对每条腿，它都会：

```
r_leg = r_body_foot[:, i] - LEG_ORIGINS[:, i]
alpha[:, i] = leg_explicit_inverse_kinematics(r_leg, i, config)
```

12.1 单腿 IK 做了什么

单腿 IK 可以分成两步：

```
先在 y-z 平面解 abduction
```

```
  -> 再在 hip-foot 平面解 hip / knee 二连杆
```

它最后返回：

```
[abduction_angle, hip_angle, knee_angle]
```

这也是为什么项目里关节角矩阵也是 `(3, 4)`：

```
rows = [abduction, hip, knee]
cols = [FR, FL, BR, BL]
```

12.2 为什么 IK 是这个仓库最关键的“桥”

因为上层控制器做的所有事情，最终都是在生成：

```
foot_locations
```

只有过了 IK 之后，才会真正进入：

```
joint_angles
```

所以如果你问：

> 该项目的控制算法输入到舵机输出在哪里被体现？

最核心的答案就是：

```
JoystickInterface -> Command
  -> Controller 生成 foot_locations
  -> Kinematics 把 foot_locations 变成 joint_angles
  -> HardwareInterface 把 joint_angles 变成 PWM
```

13. 硬件输出：`joint_angles` 怎么变成 PWM

`pupper/HardwareInterface.py` 这一层就是最终执行层。

它的链路非常清晰：

```
joint_angle
  -> angle_to_pwm()
  -> pwm_to_duty_cycle()
  -> pigpio.set_PWM_dutycycle()
```

13.1 从角度到 PWM 的公式

核心变换是：

```
angle_deviation =
  (angle - neutral_angle) * servo_multiplier

pulse_width_micros =
  neutral_position_pwm + micros_per_rad * angle_deviation
```

然后再把脉宽转成 duty cycle。

这里你能看到 3 个非常工程化的补偿量：

- `neutral_angle_degrees`
- `servo_multipliers`
- `micros_per_rad`

这三个量决定了：

- 你的理论关节角
- 如何被映射成真实舵机方向和真实中位

13.2 每条腿符号为什么不一样

`servo_multipliers` 里存在正负号翻转：

```
[
 [ 1, 1, 1, 1],
 [-1, 1, -1, 1],
 [ 1, -1, 1, -1],
]
```

这很典型，原因是：

- 左右腿舵机安装镜像
- 上下关节零位方向不同
- 实际机械装配决定了符号约定必须按腿翻转

13.3 标定文件是机器生成的

`pupper/ServoCalibration.py` 里当前能看到：

```
MICROS_PER_RAD
NEUTRAL_ANGLE_DEGREES
```

这个文件不是手写配置，而是由 `calibrate_servos.py` 覆盖写出的。

所以如果你改了标定流程，不应该直接手改最终文件，而应该：

```
先改标定逻辑
 -> 再运行标定脚本
 -> 再让脚本写回 ServoCalibration.py
```

而且 `calibrate_servos.py` 里整套流程非常“ 人工介入式 ”：

```
逐电机移动
 -> 人看连杆是否到水平 / 45 度
 -> 键盘微调 offset
 -> 再把 offset 写回标定文件
```

这再次说明仓库的工程定位是：

- 教学友好

- 动手调试友好
- 不是高度自动化生产线式标定

14. 配置：这个项目几乎所有行为都被 `Configuration` 控着

`pupper/Config.py` 是整个仓库最值得反复读的配置文件。

它几乎把整个项目分成几类参数：

- 命令范围
- 机身姿态调节参数
- 站姿默认几何
- swing 轨迹参数
- gait 时序参数
- 机器人几何参数
- 机器人惯性参数

14.1 如果你只想让机器人“走起来”，先看这些参数

```
max_x_velocity
max_y_velocity
max_yaw_rate
default_z_ref
z_clearance
overlap_time
swing_time
delta_x
delta_y
```

14.2 如果你想让它“走得更稳”，优先看这些参数

```
z_time_constant
pitch_deadband
pitch_time_constant
max_pitch_rate
max_stance_yaw
max_stance_yaw_rate
alpha
beta
```

14.3 如果你想让 IK 和机械结构对齐，先看这些参数

```
LEG_FB
LEG_LR
LEG_L1
LEG_L2
ABDUCTION_OFFSET
LEG_ORIGINS
ABDUCTION_OFFSETS
```

也就是说：

```
步态调不动
 -> 先看 gait / swing / stance 参数

脚落点不对
 -> 再看 default_stance / geometry

角度正确但电机动作不对
 -> 最后看 ServoCalibration / servo_multipliers
```

15. 部署与外部依赖：真正跑实机时还缺哪些东西

`install.sh` 说明了一件很重要的事：

这个仓库不是一个“自带全部依赖”的单仓项目。

它还会拉两个外部仓库：

- `PupperCommand`
- `PS4Joystick`

而且还依赖：

- `pigpio`
- `pyserial`
- `transforms3d`
- `UDPComms`

从阅读角度上说，`JoystickInterface` 这层其实默认依赖了一条仓外通信链：

```
PS4 手柄
 -> 外部 joystick.py
 -> UDP
 -> JoystickInterface
 -> Command
```

也就是说：

- 本仓库并不直接读蓝牙手柄
- 它读的是已经被外部程序整理好的 UDP 消息

所以从工程上说，`StanfordQuadruped` 更像：

```
机器人主体控制仓库
 + 手柄输入仓库
 + 系统服务配置
 + pigpio 舵机输出
```

`robot.service` 则说明它原本的预期部署方式是：

```
开机后先起 joystick.service
再起 robot.service
由 systemd 常驻运行 run_robot.py
```

这对读代码也有帮助：

- 你看到的 `run_robot.py` 不是测试脚本
- 它是被当成真正的常驻机器人服务来设计的

16. 这个项目最适合怎样继续改

如果你准备继续在这个仓库上做开发，最稳妥的切入顺序是：

```
先改输入层
 -> 再改 gait / foot target
 -> 再改 IK / 几何
 -> 最后改硬件映射
```

16.1 想接导航或高层策略

最自然的入口是：

- 复用 `Command`
- 把手柄输入换成自主规划输入

也就是让：

```
JoystickInterface -> Command
```

变成：

```
Planner / Policy -> Command
```

16.2 想接 MuJoCo / Gazebo

最自然的入口是：

- 保留 `Controller`
- 替换 `HardwareInterface`

也就是：

```
保留 foot planning + IK
 -> 把 joint_angles 输出到仿真电机接口
```

16.3 想加更复杂控制器

最自然的升级路径是：

```
先保留当前状态机
 -> 把 StanceController 升级
 -> 把 SwingController 升级
 -> 再考虑把 IK-only 架构升到 torque / QP / WBC
```

不要一上来就把所有层全推倒。

17. 代码阅读时最该注意的几个“ 项目特点 ”

17.1 它是强几何、弱动力学的项目

核心变量长期围绕：

```
foot_locations
joint_angles
default_stance
touchdown_location
smoothed_yaw
quat_orientation
```

而不是：

```
mass matrix
contact force
qp solution
```

17.2 它非常适合教学，但也带着历史痕迹

比如：

- `run_robot.py` 末尾直接调用 `main()`
- `State` 里有动态补写字段
- `Controller.set_pose_to_default()` 里存在未清理的旧引用
- `src/Tests.py` 明显还带着旧版/历史测试痕迹
- `woofer/` 支线和 `pupper/` 主线共存在一个仓库中

所以读代码时要分清：

```
哪些是主干运行路径
 -> 哪些是历史遗留或辅助脚本
```

特别是 `Controller.set_pose_to_default()` 这一点，值得明确指出：

```
state.foot_locations = ...
state.joint_angles = controller.inverse_kinematics(...)
```

这里直接引用了未在函数参数中传入的 `state` 和 `controller` 名字。

这说明它不是当前主路径里的可靠接口，更像一个没有继续维护的遗留辅助方法。

17.3 真正的主干文件没有很多

这个项目真正的主干文件其实就是：

```
run_robot.py
 -> src/JoystickInterface.py
```

-> src/Controller.py
-> src/Gaits.py
-> src/StanceController.py
-> src/SwingLegController.py
-> pupper/Kinematics.py
-> pupper/HardwareInterface.py
-> pupper/Config.py

你把这 8 个点看透，项目主线就已经掌握了。

18. 和通用四足控制教程怎么对照着读

如果你已经有 `docs/quadruped_motion_control_tutorial/quadruped_motion_control_tutorial.pdf` 的背景，可以直接做下面这张映射：

通用教程里的 “ 命令层 ”
-> StanfordQuadruped 的 JoystickInterface / Command

通用教程里的 “ 模式层 ”
-> StanfordQuadruped 的 BehaviorState / transition mapping

通用教程里的 “ 步态调度 ”
-> StanfordQuadruped 的 GaitController

通用教程里的 “ 摆动相 / 支撑相 ”
-> StanfordQuadruped 的 SwingController / StanceController

通用教程里的 “ IK 桥接层 ”
-> StanfordQuadruped 的 four_legs_inverse_kinematics

通用教程里的 “ 执行器输出 ”
-> StanfordQuadruped 的 HardwareInterface

也就是说，这个仓库非常适合拿来做：

理论教程
-> 项目代码映射
-> 自己动手改一层
-> 再回到理论看为什么这样设计

19. 最后给一个最稳的项目阅读顺序

如果你准备从零开始读这个项目，我推荐下面这条顺序：

第 1 遍：
run_robot.py
-> Controller.py

第 2 遍：
Gaits.py
-> StanceController.py

-> SwingLegController.py

第 3 遍：
Kinematics.py
-> HardwareInterface.py
-> Config.py

第 4 遍：
JoystickInterface.py
-> calibrate_servos.py
-> install.sh / robot.service

如果你是准备继续开发，而不是只读懂，那么建议再加一轮：

先记录一遍 Command / State / foot_locations / joint_angles 的流向
-> 再决定你要改输入、改步态、改 IK 还是改硬件后端

20. 先把四个核心变量的流向记清楚

这一节很关键，因为很多人一上来就直接改 `Gaits.py` 或 `JoystickInterface.py`，结果改了半天发现自己改错层了。

这个项目最稳的理解方式，不是先问“我要改哪个文件”，而是先问：

Command 从哪里来
-> State 在哪里被改
-> foot_locations 在哪里生成
-> joint_angles 在哪里落到硬件
-> 然后再决定改输入、改步态、改 IK 还是改舵机后端

20.1 总链路先看一遍

外部手柄 / joystick.py
-> UDPComms
-> JoystickInterface.get_command(state)
-> Command
-> run_robot.py 主循环
-> Controller.run(state, command)
-> foot_locations
-> rotated_foot_locations
-> four_legs_inverse_kinematics()
-> joint_angles
-> HardwareInterface.set_actuator_postions()
-> angle_to_pwm()
-> pwm_to_duty_cycle()
-> pigpio.set_PWM_dutycycle()

对应主循环其实就这么几行：

```
command = joystick_interface.get_command(state)
state.quat_orientation = imu.read_orientation() if use_imu else np.array([1, 0, 0, 0])
controller.run(state, command)
```

```
hardware_interface.set_actuator_postions(state.joint_angles)
```

所以这四个量不是并列关系，而是一条很明确的因果链：

```
Command
 -> 影响 foot_locations
 -> foot_locations 经过 IK 变成 joint_angles
 -> joint_angles 输出到舵机

State
 -> 给 Controller 提供 " 上一拍记忆 "
 -> 也承接 Controller 这一拍算出来的结果
```

20.2 `Command` 的流向

`Command` 是 " 这一拍想让机器人怎么动 "，它的出生点非常明确：`src/JoystickInterface.py`。

```
UDP 手柄消息
 -> JoystickInterface.get_command(state)
 -> 新建 Command()
 -> 填 discrete event
 -> 填 continuous command
 -> 返回给 run_robot.py
 -> 传给 Controller.run(state, command)
```

`get_command()` 里主要做了两类事：

- 离散事件：
- `L1 -> activate_event`
- `R1 -> trot_event`
- `x -> hop_event`
- 连续量：
- `ly -> x_vel`
- `lx -> y_vel`
- `rx -> yaw_rate`
- `ry -> pitch`
- `dpady -> body height`
- `dpadx -> roll`

然后 `Controller.run()` 会在几个位置消费这些命令：

```
activate_event / trot_event / hop_event
 -> behavior_state 切换

horizontal_velocity / yaw_rate / height
 -> StanceController.position_delta()

horizontal_velocity / yaw_rate / height
```

```
  -> SwingController.raibert_touchdown_location()
  -> SwingController.next_foot_location()

roll / pitch / yaw_rate
  -> Controller.run() 里的机身姿态旋转或 yaw 平滑
```

这里有一个很重要的边界：

- `Command` 是本拍输入
- 它不是长期状态容器
- 代码里也没有把整份 `Command` 原样塞回 `state`

当前主链路里，只有 `command.pitch`、`command.roll`、`command.height` 会在 `Controller.run()` 尾部写回 `state`，作为下一拍滤波参考；`horizontal_velocity`、`yaw_rate` 并不会被完整持久化进 `state`。

所以：

```
如果你要改手柄映射 / 改 Planner 输出格式
  -> 先改 JoystickInterface.py
  -> 必要时补 Command.py 字段

不要一上来就改 Gaits.py
  -> 因为那已经是输入之后的步态执行层
```

20.3 `State` 的流向

`State` 是“机器人现在记住了什么”。它在 `src/State.py` 里初始化，但真正的写入分散在主循环和控制器内部。

```
State()
  -> 初始 behavior_state / ticks / foot_locations / joint_angles
  -> run_robot.py 每拍写 quat_orientation
  -> Controller.run() 里更新 behavior_state
  -> Controller.run() 里更新 foot_locations / joint_angles
  -> Controller.run() 里更新 ticks / pitch / roll / height
```

你可以把它理解成“控制器的持久化记忆区”。

几个关键点要特别记住：

- `state.quat_orientation` 不是在 `State.__init__()` 里声明的
- 它是在 `run_robot.py` 主循环中动态写进去的
- `state.foot_locations` 与 `state.joint_angles` 才是每拍闭环真正持续往前传的核心结果
- `state.ticks` 是 gait 相位推进的时间基准

还有一个很容易忽略的事实：

- `State.py` 里有 `horizontal_velocity`、`yaw_rate`、`activation`
- 但当前主链路几乎没有真正使用它们

- 这些更像早期接口遗留或未彻底清理的状态字段

这意味着：

```
如果你要加一个新输入量
 -> 先判断它是 “ 瞬时命令 ” 还是 “ 持久状态 ”

瞬时命令
 -> 优先加在 Command

持久记忆
 -> 再考虑是否放进 State
```

20.4 `foot_locations` 的流向

`foot_locations` 是这个项目里最核心的中间量。它表示 “ 机身坐标系下，四只脚此刻应该去哪里 ” 。

在 `State` 里它一开始只是一个 `(3, 4)` 零矩阵，但真正有意义的内容来自 `Controller.run()`。

```
state.foot_locations
 -> Controller.step_gait()
 -> gait_controller.contacts(state.ticks)
 -> 每条腿分流
 -> 支撑腿走 StanceController.next_foot_location()
 -> 摆动腿走 SwingController.next_foot_location()
 -> 得到 new_foot_locations
 -> 写回 state.foot_locations
```

具体展开是：

```
contact_modes = gait_controller.contacts(state.ticks)
for each leg:
  if contact_mode == 1:
    new_location = stance_controller.next_foot_location(...)
  else:
    new_location = swing_controller.next_foot_location(...)
state.foot_locations = new_foot_locations
```

然后要注意第二层处理：

```
state.foot_locations
 -> 先作为 “ 名义足端目标 ”
 -> 在 TROT / REST 下再做机身姿态旋转
 -> 得到 rotated_foot_locations
 -> IK 实际吃的是 rotated_foot_locations
```

也就是说，在 `TROT` 模式下：

- `state.foot_locations` 保存的是 gait / stance / swing 算出来的名义足端目标
- `rotated_foot_locations` 才是加入 `command.roll`、`command.pitch`、IMU 倾斜补偿之后送去 IK 的那一版
- `contact_modes` 会被算出来，但当前没有长期保存进 `state`

这几个边界特别重要，因为它们决定你应该改哪里：

```
想改接触时序 / 哪条腿先落地
  -> 改 Gaits.py

想改支撑腿怎么 " 跟着机身走 "
  -> 改 StanceController.py

想改摆动腿落脚点 / 抬脚轨迹 / 摆腿高度
  -> 改 SwingLegController.py

想改机身姿态补偿
  -> 改 Controller.py
```

20.5 `joint_angles` 的流向

`joint_angles` 是控制器给硬件层的最后一个 " 几何输出 " 。

它不是步态层直接生成的，而是 IK 把足端目标解算之后得到的：

```
foot_locations 或 rotated_foot_locations
  -> four_legs_inverse_kinematics()
  -> state.joint_angles
  -> HardwareInterface.set_actuator_postions()
```

在 `pupper/Kinematics.py` 里：

```
four_legs_inverse_kinematics()
  -> 对四条腿逐列处理
  -> leg_explicit_inverse_kinematics()
  -> abduction / hip / knee 三个角
  -> 拼成 (3, 4) joint_angles
```

到了 `pupper/HardwareInterface.py` 里，这些角度继续走下面这条链：

```
joint_angles
  -> angle_to_pwm()
  -> angle_to_duty_cycle()
  -> pwm_to_duty_cycle()
  -> pi.set_PWM_dutycycle()
```

所以 `joint_angles` 的语义是：

- 它已经不是 " 脚该去哪里 "
- 而是 " 每个关节该转多少 "
- 再往后就是舵机标定、PWM、GPIO 输出问题了

因此：

```
想改腿长、几何解算、关节定义
  -> 改 Kinematics.py / Config.py

想改舵机中位、方向、脉宽映射
  -> 改 HardwareInterface.py / ServoCalibration.py
```

```
不要把这些修改混进 gait 文件里
```

20.6 先判断你到底该改“输入”还是改“步态”

把上面的流向合起来之后，决策会非常清楚：

```
外部输入变化
 -> 先看 Command

落脚节奏变化
 -> 先看 Gaits

足端轨迹变化
 -> 先看 Stance / Swing

关节解算变化
 -> 先看 IK

舵机输出变化
 -> 先看 HardwareInterface
```

更直白一点：

- 你想把摇杆、键盘、policy、导航器输出接进来，本质上是在改 `Command` 输入层
- 你想让 trot 更快、更稳、抬脚更高、落脚更前，本质上是在改 `foot_locations` 的生成层
- 你想让关节角定义、腿长、舵机零位变掉，本质上是在改 `joint_angles` 及其后端

所以真正稳妥的开发顺序是：

```
先画清楚变量流向
 -> 再锁定改动层级
 -> 每次只改一层
 -> 回归验证
```

这样就不容易出现“明明想改手柄输入，却把 gait 和 IK 一起改乱”的问题。

附录 A：最短路径版总结

如果把整个仓库压成 6 句话，可以这样记：

- `run_robot.py` 是系统入口
- `JoystickInterface` 把 UDP 手柄消息变成 `Command`
- `Controller.run()` 根据 `BehaviorState` 决定当前该站立、trotting 还是 hop
- `GaitController + StanceController + SwingController` 负责生成四条腿的足端目标
- `four_legs_inverse_kinematics()` 把足端目标变成四条腿的关节角
- `HardwareInterface` 把关节角变成 PWM，占空比最终送到舵机

附录 B：你读这个项目时最该盯住的变量

command.horizontal_velocity
command.yaw_rate
state.behavior_state
state.foot_locations
state.joint_angles
contact_modes
default_stance
touchdown_location
smoothed_yaw
quat_orientation

附录 C：这个项目最适合拿来学什么

学四足项目主循环怎么写
-> 学 gait / stance / swing 如何拆层
-> 学 IK 在四足项目里如何充当桥梁
-> 学 position-level 舵机控制链路怎样落地